

UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA,
ELECTRÓNICA Y SISTEMAS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



TRABAJO ENCARGADO

PRÁCTICA N.º 04: ARREGLOS DINÁMICOS, VECTORES Y MATRICES

CURSO:

ALGORITMOS Y ESTRUCTURAS DE DATOS (SIS210)

DOCENTE:

Dr. ZANABRIA GÁLVEZ ALDO HERNÁN

PRESENTADO POR:

1. PARILLO UMIÑA JHONATAN
2. RODRIGUEZ AGUILAR ALAIN NELSON
3. CHOQUE GALVEZ LUIGGI LEONEL YEFRY
4. PANDIA QUISPE MELVIN MARCONI

LINK DE DRIVE:

https://drive.google.com/drive/folders/16Jat7jbL_OLYaSeAU9VwghLYJr96UUbC?usp=sharing

SEMESTRE: IV

PUNO – PERÚ
2026



ÍNDICE

1. Introducción y objetivos.....	5
1.1 Objetivo general.....	5
1.2 Objetivos específicos.....	5
2. Marco teórico.....	5
2.1 Fórmula de dirección de un arreglo.....	5
2.2 Análisis amortizado del arreglo dinámico.....	5
2.3 Row-major y localidad de caché.....	6
2.4 Estructuras comparadas.....	6
3. Materiales y entorno experimental.....	6
4. Actividad 1: Python — acceso $O(1)$ vs. inserción al inicio $O(n)$	7
4.1 Código utilizado.....	7
4.2 Resultados.....	9
4.3 Análisis.....	9
4.4 Verificación del acceso $O(1)$	10
5. Actividad 2: C++ — <code>std::vector</code> , redimensionamiento y <code>capacity()</code>	10
5.1 Código utilizado.....	10
5.2 Redimensionamientos registrados.....	13
5.3 Benchmark: <code>push_back</code> con y sin <code>reserve()</code>	14
5.4 Análisis.....	14
6. Actividad 3: C++ — impacto de la localidad de caché en matrices.....	14
6.1 Código utilizado.....	14
6.2 Resultados.....	16
6.3 Análisis: cómo crece el impacto.....	17
7. Actividad 4: Python — NumPy y operaciones matriciales.....	18
7.1 Código utilizado.....	18
7.2 Resultados.....	19
7.3 Análisis de la razón de velocidad.....	19
8. Preguntas de análisis.....	20
8.1 ¿Cuál es el ratio exacto de crecimiento de la capacidad? ¿Es siempre $2x$?.....	20
8.2 ¿Por qué el Experimento 3 necesita la variable suma? ¿Cómo se comprueba que el benchmark es real?.....	21
8.3 ¿Cómo logra <code>std::deque</code> $O(1)$ en ambos extremos sin ser contiguo? ¿Cuál es su desventaja?.....	21
8.4 ¿Cuándo se justifica <code>collections.deque</code> en lugar de <code>list</code> ? Caso de uso del curso.....	22
9. Trabajo de investigación: algoritmo de Strassen.....	22
9.1 Idea del algoritmo.....	22



9.2 Aplicación del Teorema Maestro	23
9.3 ¿A partir de qué tamaño supera al algoritmo $O(n^3)$?	23
9.4 Limitaciones prácticas	26
10. Discusión de limitaciones	26
11. Conclusiones.....	26
12. Declaración de uso de inteligencia artificial.....	27
13. Referencias.....	27

1. Introducción y objetivos

Las estructuras contiguas —arreglos estáticos, vectores dinámicos y matrices— son la base de casi todas las demás estructuras de datos. Su principal fortaleza es el acceso por índice en tiempo constante, pero su costo real depende de dos factores que la notación asintótica no muestra por sí sola: la estrategia de **redimensionamiento** del arreglo dinámico y la **localidad de caché** con la que se recorre la memoria (Cormen et al., 2022). En esta práctica se verifican ambos aspectos de forma experimental en Python 3.12 y C++17, siguiendo las cuatro actividades de la guía y completando la investigación sobre el algoritmo de Strassen.

1.1 Objetivo general

Comprender la representación en memoria de arreglos estáticos, vectores dinámicos y matrices, analizar el costo de sus operaciones (acceso, inserción, eliminación y redimensionamiento) y verificar experimentalmente el análisis amortizado del arreglo dinámico y el impacto de la localidad de caché en el acceso a matrices.

1.2 Objetivos específicos

- Verificar que el acceso `arr[i]` es $O(1)$ y que la inserción al inicio es $O(n)$, midiendo tiempos reales en Python.
- Demostrar el comportamiento amortizado de `push_back` registrando `capacity()` en cada redimensionamiento de `std::vector`.
- Cuantificar el impacto de la localidad de caché (recorrido por filas frente a columnas) en matrices de 100×100 a 2000×2000 .
- Comparar `std::vector`, `std::deque`, `list` y `collections.deque` mediante un benchmark reproducible.
- Analizar el algoritmo de Strassen con el Teorema Maestro y determinar empíricamente su punto de inflexión frente al algoritmo $O(n^3)$.

2. Marco teórico

2.1 Fórmula de dirección de un arreglo

Un arreglo ocupa un bloque contiguo de memoria. La dirección de cualquier elemento se obtiene con una multiplicación y una suma, sin recorrer los elementos anteriores; por eso el acceso por índice es $O(1)$ (Cormen et al., 2022):

$$\text{Dirección}(\text{arr}[i]) = \text{dirección_base} + i \times \text{sizeof}(T)$$

Por ejemplo, para `int arr[5]` con dirección base `0x1000` y `sizeof(int) = 4`, el elemento `arr[3]` está en $0x1000 + 3 \times 4 = 0x100C$. En una matriz almacenada por filas (row-major) de C columnas, la fórmula se extiende a `base + (i × C + j) × sizeof(T)`.

2.2 Análisis amortizado del arreglo dinámico

Cuando el vector está lleno (`size == capacity`), reserva un bloque mayor —el doble en GCC— y copia todos los elementos. Esa inserción cuesta $O(n)$, pero ocurre cada vez con menos frecuencia. Si se parte de capacidad 1 y se duplica, el total de copias tras n inserciones es:

$$1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1 < 2n$$

Por tanto, el costo total de n inserciones es menor que $n + 2n = 3n$, y el costo **amortizado** por inserción es $O(1)$. En el *método del banco*, cada inserción paga 3 unidades: 1 por escribir su elemento y 2 que deposita como crédito; cuando la capacidad pasa de m a $2m$, las $m/2$ inserciones posteriores a la duplicación anterior han acumulado exactamente m unidades, suficientes para copiar los m elementos (Cormen et al., 2022; Goodrich et al., 2011).

2.3 Row-major y localidad de caché

C y C++ almacenan las matrices por filas: los elementos de una misma fila son contiguos. El procesador no lee bytes sueltos, sino **líneas de caché** de 64 bytes; con `long long` (8 bytes) cada línea contiene 8 elementos. Recorrer por filas aprovecha los 8 elementos de cada línea cargada y permite que el prefetcher del hardware anticipe los accesos. Recorrer por columnas salta $N \times 8$ bytes entre accesos consecutivos, de modo que casi cada acceso toca una línea (y con frecuencia una página) distinta. La complejidad es $\Theta(N^2)$ en ambos casos: lo que cambia es la constante oculta (Bryant & O'Hallaron, 2016).

2.4 Estructuras comparadas

Tabla 1

Costo de las operaciones principales en las estructuras estudiadas

Estructura	Acceso [i]	Insertar al final	Insertar al inicio	Memoria
<code>std::vector / list</code> (Python)	$O(1)$	$O(1)$ amortizado	$O(n)$	Contigua
<code>std::deque</code>	$O(1)$ (doble indirección)	$O(1)$ amortizado	$O(1)$ amortizado	Bloques + mapa
<code>collections.deque</code>	$O(1)$ extremos, $O(n)$ centro	$O(1)$	$O(1)$	Lista de bloques
Arreglo estático <code>T a[n]</code>	$O(1)$	No aplica	No aplica	Contigua, tamaño fijo

Nota. Elaboración propia a partir de cpreference.com (2024) y Python Software Foundation (2026).

3. Materiales y entorno experimental

Todas las mediciones se realizaron en el mismo equipo, sin otras cargas de trabajo, siguiendo las instrucciones de la guía (compilación con `-O2`). Cada medición se repitió varias veces y se reporta la **mediana**, que es menos sensible que el promedio a interrupciones del sistema operativo.

Tabla 2

Entorno de ejecución de los experimentos

Componente	Especificación
Procesador	Intel Xeon @ 2.10 GHz (1 núcleo virtual)
Caché	L1d 48 KiB · L2 2 MiB · L3 260 MiB (compartida, reportada por la VM)
Sistema operativo	Ubuntu 24.04 LTS (Linux, 64 bits)
Compilador C++	g++ 13.3.0 · -std=c++17 -O2 -Wall · libstdc++
Python	CPython 3.12.3 · time.perf_counter
NumPy	NumPy 2.4.4 con OpenBLAS 0.3.31
Repeticiones	Exp. 1: 5 · Exp. 2: 11 · Exp. 3: 9 · Exp. 4: 50 (NumPy) y 1–3 (puro) · Strassen: 1–9
Estadístico reportado	Mediana de las repeticiones

Los archivos entregados en la carpeta de Drive son `Practica_S04_Arreglos_ApellidoNombre.ipynb` (actividades 1 y 4) y `Practica_S04_Arreglos_ApellidoNombre.cpp` (actividades 2 y 3), además de `strassen.cpp` para la investigación.

4. Actividad 1: Python — acceso $O(1)$ vs. inserción al inicio $O(n)$

4.1 Código utilizado

Se tomó el código de la guía y se añadieron tres elementos: 5 repeticiones con mediana, una tercera serie con `collections.deque.appendleft` (que servirá para la pregunta 8.4) y la verificación del acceso `lst[i]` en tres posiciones. A continuación se transcribe el programa completo tal como se ejecutó (`actividad1.py`).

```
# =====
# Practica N.o 04 - ACTIVIDAD 1 (Python)
# Acceso O(1) vs insercion al inicio O(n)
# =====
import time
import statistics
from collections import deque
import matplotlib.pyplot as plt

ns = [1_000, 5_000, 10_000, 50_000, 100_000]
R = 5                                # repeticiones por tamaño
tiempos_append, tiempos_insert0, tiempos_deque = [], [], []

for n in ns:
    ta, ti, td = [], [], []
    for _ in range(R):
        # --- append: O(1) amortizado ---
        lst = []
        t0 = time.perf_counter()
        for i in range(n):
            lst.append(i)
```

```
ta.append((time.perf_counter() - t0) * 1000)

# --- insert(0, x): O(n) por insercion ---
lst = []
t0 = time.perf_counter()
for i in range(n):
    lst.insert(0, i)
ti.append((time.perf_counter() - t0) * 1000)

# --- deque.appendleft: O(1) (para la pregunta 8.4) ---
d = deque()
t0 = time.perf_counter()
for i in range(n):
    d.appendleft(i)
td.append((time.perf_counter() - t0) * 1000)

tiempos_append.append(statistics.median(ta))
tiempos_insert0.append(statistics.median(ti))
tiempos_deque.append(statistics.median(td))

print(f"{'n':>10} {'append (ms)':>15} {'insert(0) (ms)':>16} {'ratio':>8}
{'deque (ms)':>12}")
for n, tapp, tins, tdq in zip(ns, tiempos_append, tiempos_insert0,
tiempos_deque):
    print(f"{'n':>10} {'tapp:>15.3f} {'tins:>16.3f} {'tins/tapp:>7.1f}x
{'tdq:>12.3f}")

# --- Verificacion del acceso O(1): lst[i] en inicio, centro y final ---
print(f"\n{'n':>10} {'lst[0]':>10} {'lst[n//2]':>12} {'lst[n-1]':>11} (ns
por acceso)")
for n in ns:
    lst = list(range(n))
    tiempos = []
    for pos in (0, n // 2, n - 1):
        t0 = time.perf_counter_ns()
        for _ in range(100_000):
            x = lst[pos]
        tiempos.append((time.perf_counter_ns() - t0) / 100_000)
    print(f"{'n':>10} {'tiempos[0]':>10.1f} {'tiempos[1]':>12.1f}
{'tiempos[2]':>11.1f}")

# --- Figura 1: escala log-log ---
plt.figure(figsize=(7, 3.6))
plt.loglog(ns, tiempos_insert0, 'o-', label='list.insert(0, x) - O(n) por
operacion')
plt.loglog(ns, tiempos_append, 's-', label='list.append(x) - O(1)
amortizado')
plt.loglog(ns, tiempos_deque, '^--', label='deque.appendleft(x) - O(1)')
```

```
plt.xlabel('n (numero de inserciones)')
plt.ylabel('Tiempo total (ms, escala log)')
plt.legend()
plt.grid(True, which='both', ls='--', alpha=0.4)
plt.tight_layout()
plt.show()
```

4.2 Resultados

Tabla 3

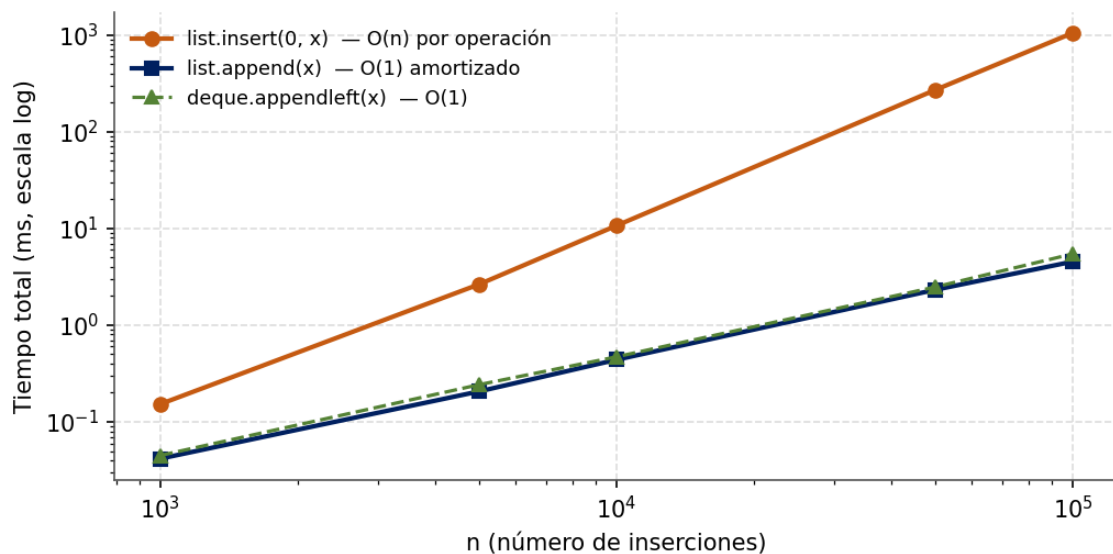
Tiempo total de n inserciones con append e insert(0) en Python

n	append N veces (ms)	insert(0) N veces (ms)	Ratio insert/append	deque.appendleft (ms)
1,000	0.042	0.153	3.7x	0.045
5,000	0.207	2.645	12.8x	0.242
10,000	0.439	10.687	24.3x	0.470
50,000	2.323	270.893	116.6x	2.482
100,000	4.538	1,050.038	231.4x	5.458

Nota. Mediana de 5 repeticiones. La fila resaltada marca el punto a partir del cual la diferencia supera un orden de magnitud.

Figura 1

Tiempo total de n inserciones (escala log-log)



Nota. En escala log-log, una recta de pendiente 1 indica crecimiento lineal del tiempo total y una de pendiente 2, crecimiento cuadrático.

4.3 Análisis

La serie de `append` crece de forma **lineal**: al multiplicar n por 100 (de 1 000 a 100 000), el tiempo total pasa de 0.042 a 4.538 ms, es decir, se multiplica por unos 108. Eso equivale a unos 45 ns por inserción, constante para todos los tamaños, tal como predice el costo $O(1)$ amortizado.

La serie de `insert(0, x)` crece de forma **cuadrática**: con el mismo aumento de 100 veces en n , el tiempo se multiplica por unos 6 860 (de 0.153 a 1 050 ms). Cada inserción desplaza todos

los elementos existentes una posición, así que el total es $0 + 1 + \dots + (n - 1) \approx n^2/2$ movimientos. El exponente observado es algo menor que 2 porque, para n pequeño, el desplazamiento lo hace `memmove` en C y el costo fijo del intérprete todavía pesa. En la Figura 1 esto se ve como una pendiente cercana a 2 frente a la pendiente 1 de `append`.

¿A partir de qué n se vuelve significativa la diferencia? Con $n = 1\,000$ el ratio es solo 3.7x y ambas operaciones tardan menos de 0.2 ms. A partir de $n \approx 5\,000$ la diferencia supera un orden de magnitud (12.8x) y sigue creciendo proporcionalmente a n : 24x con 10 000, 117x con 50 000 y 231x con 100 000. Con 100 000 elementos, insertar al inicio tarda más de un segundo, mientras que `append` no llega a 5 ms. El ratio crece de forma aproximadamente lineal porque es el cociente de un costo $\Theta(n^2)$ entre uno $\Theta(n)$.

4.4 Verificación del acceso $O(1)$

Para completar el primer objetivo específico se midió el tiempo de `lst[i]` en tres posiciones (inicio, centro y final) con 100 000 lecturas cada una.

Tabla 4

Tiempo por acceso `lst[i]` en nanosegundos, según el tamaño de la lista y la posición

n	lst[0]	lst[n//2]	lst[n-1]
1,000	54.5	55.5	56.2
10,000	53.4	55.7	55.8
100,000	53.7	58.1	55.9

Nota. El valor incluye el costo del bucle `for` de Python, que es el mismo en todas las celdas.

El tiempo se mantiene entre 53 y 58 ns sin importar n ni la posición. Esto confirma que el acceso por índice no depende del tamaño de la lista: es $O(1)$, coherente con la fórmula de dirección de la sección 2.1.

5. Actividad 2: C++ — `std::vector`, redimensionamiento y `capacity()`

5.1 Código utilizado

Se ejecutó el código de la guía con tres añadidos: un contador de copias (en cada redimensionamiento se copian los `size()` elementos que había antes del `push_back`), la mediana de 11 repeticiones en el benchmark de reserva y una comparación de acceso aleatorio entre `vector` y `deque` para la pregunta 8.3. El programa completo es el siguiente (`actividad2.cpp`).

```
// =====  
// Practica N.o 04 - ACTIVIDAD 2 (C++17)  
// std::vector: redimensionamiento, capacity() y reserve()  
// Compilar: g++ -std=c++17 -O2 -Wall -o act2 actividad2.cpp && ./act2  
// =====  
  
#include <vector>  
#include <deque>  
#include <iostream>
```

```
#include <chrono>
#include <algorithm>
using Clock = std::chrono::high_resolution_clock;

double mediana(std::vector<double> v) {
    std::sort(v.begin(), v.end());
    return v[v.size() / 2];
}

double ms_desde(Clock::time_point t0) {
    return std::chrono::duration<double, std::milli>(Clock::now() - t0).count();
}

int main() {
    // ----- 1. Momentos de redimensionamiento -----
    std::vector<int> v;
    v.reserve(0); // empezar sin pre-reservar
    std::cout << "Momento de redimensionamiento:\n";
    std::cout << " #   size   capacity   ratio   copias\n";
    size_t cap_anterior = 0;
    unsigned long long copias = 0;
    int redim = 0;
    for (int i = 0; i < 1'000'000; i++) {
        size_t antes = v.size(); // elementos que habra que copiar
        v.push_back(i);
        if (v.capacity() != cap_anterior) {
            copias += antes;
            redim++;
            std::cout << "   " << redim
                      << "   size=" << v.size()
                      << "   capacity=" << v.capacity()
                      << "   ratio=" << (cap_anterior > 0
                                         ? (double)v.capacity() / cap_anterior : 0)
                      << "   copias=" << antes << '\n';
            cap_anterior = v.capacity();
        }
    }
    std::cout << "Redimensionamientos: " << redim
              << "   Copias totales: " << copias
              << "   (2n = " << 2 * 1'000'000 << ")\n\n";

    // ----- 2. Benchmark: push_back vs pre-reservar -----
    const int N = 1'000'000, R = 11; // R repeticiones -> mediana
    std::vector<double> sin_reserva, con_reserva, deque_front;
    for (int r = 0; r < R; r++) {
        std::vector<int> v1;
        auto t0 = Clock::now();
        for (int i = 0; i < N; i++) v1.push_back(i);
        sin_reserva.push_back(ms_desde(t0));
    }
```

```
std::vector<int> v2;
v2.reserve(N); // evita todos los redimensionamientos
t0 = Clock::now();
for (int i = 0; i < N; i++) v2.push_back(i);
con_reserva.push_back(ms_desde(t0));

std::deque<int> d; // referencia para la pregunta 8.3
t0 = Clock::now();
for (int i = 0; i < N; i++) d.push_front(i);
deque_front.push_back(ms_desde(t0));
}

double ms_sin = mediana(sin_reserva), ms_con = mediana(con_reserva);
std::cout << "Sin reserva: " << ms_sin << " ms\n";
std::cout << "Con reserve(): " << ms_con << " ms\n";
std::cout << "Aceleracion: " << ms_sin / ms_con << "x\n";
std::cout << "deque::push_front: " << mediana(deque_front) << " ms\n\n";

// ----- 3. Acceso aleatorio: vector vs deque (pregunta 8.3) -----
std::vector<int> vv(N);
std::deque<int> dd(N);
for (int i = 0; i < N; i++) { vv[i] = i; dd[i] = i; }
std::vector<int> idx(N);
unsigned s = 12345;
for (int i = 0; i < N; i++) { s = s * 1103515245u + 12345u; idx[i] = (s >> 8) %
N; }

std::vector<double> t_vec, t_deq;
long long suma = 0; // se imprime para que no se elimine el bucle
for (int r = 0; r < R; r++) {
    auto t0 = Clock::now();
    for (int i = 0; i < N; i++) suma += vv[idx[i]];
    t_vec.push_back(ms_desde(t0));

    t0 = Clock::now();
    for (int i = 0; i < N; i++) suma += dd[idx[i]];
    t_deq.push_back(ms_desde(t0));
}

std::cout << "Acceso aleatorio vector: " << mediana(t_vec) << " ms\n";
std::cout << "Acceso aleatorio deque: " << mediana(t_deq) << " ms\n";
std::cout << "Ratio deque/vector: " << mediana(t_deq) / mediana(t_vec) <<
"x\n";
std::cout << "(suma de control: " << suma << ")\n";
return 0;
}
```

5.2 Redimensionamientos registrados

Tabla 5

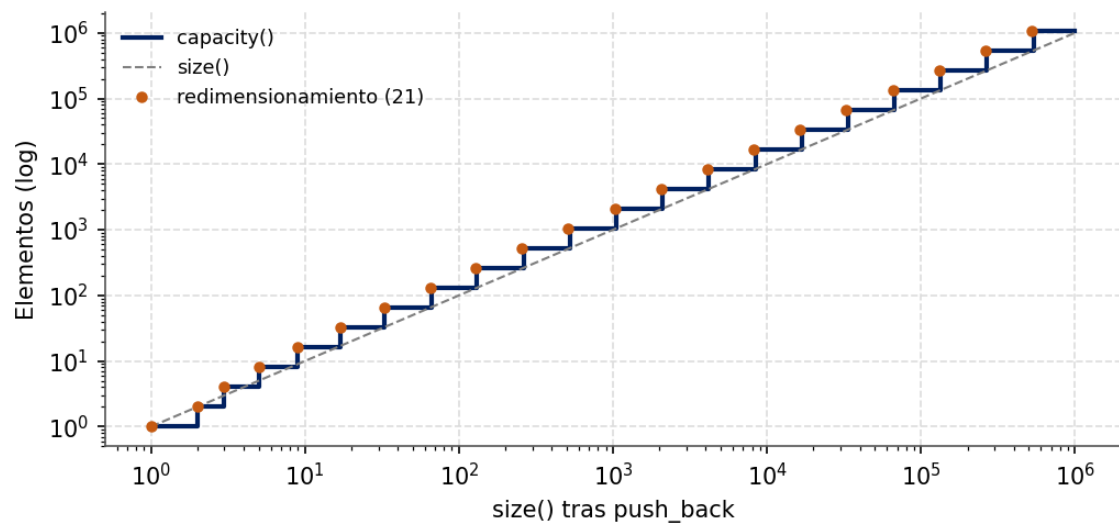
Momentos de redimensionamiento de `std::vector<int>` (GCC 13.3, libstdc++)

#	size	capacity	ratio	Copias	Acumulado
1	1	1	—	0	0
2	2	2	2.0	1	1
3	3	4	2.0	2	3
4	5	8	2.0	4	7
5	9	16	2.0	8	15
6	17	32	2.0	16	31
7	33	64	2.0	32	63
8	65	128	2.0	64	127
9	129	256	2.0	128	255
10	257	512	2.0	256	511
11	513	1,024	2.0	512	1,023
12	1,025	2,048	2.0	1,024	2,047
13	2,049	4,096	2.0	2,048	4,095
14	4,097	8,192	2.0	4,096	8,191
15	8,193	16,384	2.0	8,192	16,383
16	16,385	32,768	2.0	16,384	32,767
17	32,769	65,536	2.0	32,768	65,535
18	65,537	131,072	2.0	65,536	131,071
19	131,073	262,144	2.0	131,072	262,143
20	262,145	524,288	2.0	262,144	524,287
21	524,289	1,048,576	2.0	524,288	1,048,575

Nota. "Copias" es el número de elementos que se trasladaron al nuevo bloque en ese redimensionamiento.

Figura 2

Evolución de `capacity()` frente a `size()` (escala log-log)



5.3 Benchmark: push_back con y sin reserve()

Tabla 6

Tiempo de 1 000 000 de inserciones según la estrategia de reserva

Variante	Tiempo (ms)	Redimensionamientos	Copias
push_back sin reserva	2.272	21	1,048,575
push_back con reserve(N)	1.812	0	0
Aceleración	1.25x	—	—
std::deque::push_front (referencia)	1.042	No copia elementos	0

Nota. Mediana de 11 repeticiones, compilado con -O2.

5.4 Análisis

¿Confirma la estrategia de duplicación? Sí. Los 21 redimensionamientos siguen exactamente la secuencia de potencias de 2 (1, 2, 4, 8, ..., 1 048 576) y los 20 ratios calculados valen exactamente 2.0. El redimensionamiento se produce justo cuando `size` supera una potencia de 2 ($size = 2^k + 1$).

¿Cuántas copias totales se realizaron? $1 + 2 + 4 + \dots + 524\,288 = 2^{20} - 1 = 1\,048\,575$ copias para 1 000 000 de inserciones, es decir, 1.05 copias por elemento insertado. Esto coincide con la cota teórica de la sección 2.2 ($copias < 2n$) y demuestra que el costo amortizado de `push_back` es $O(1)$: aunque el último redimensionamiento copió por sí solo 524 288 elementos, esa operación costosa se "paga" con las 524 288 inserciones baratas que la precedieron.

El precio de esta estrategia es la memoria: al final hay capacidad para 1 048 576 enteros y solo se usan 1 000 000 (4.9 % sin usar); en el peor caso, justo después de duplicar, casi la mitad del bloque está vacía. Con `reserve(N)` se eliminan los 21 redimensionamientos y el millón de copias, lo que da una aceleración de **1.25x**. La mejora es moderada porque copiar enteros con `memmove` es muy rápido; con objetos cuya copia es costosa (por ejemplo, `std::string`), la diferencia sería mayor.

6. Actividad 3: C++ — impacto de la localidad de caché en matrices

6.1 Código utilizado

El código de la guía usa un tamaño fijo $N = 1000$. Para completar la tabla se convirtió N en un bucle sobre los cuatro tamaños pedidos y se añadió 4000×4000 como caso adicional. También se agregó una variante con un único `vector<long long>` de $N \cdot N$ elementos, contiguo en su totalidad, para comprobar si el `vector<vector>` de la guía afecta al resultado. El programa completo es el siguiente (actividad3.cpp).

```
// =====  
// Practica N.º 04 - ACTIVIDAD 3 (C++17)  
// Impacto de la localidad de cache: recorrido por filas vs por columnas  
// Compilar: g++ -std=c++17 -O2 -Wall -o act3 actividad3.cpp && ./act3  
// =====  
#include <vector>
```

```
#include <chrono>
#include <iostream>
#include <algorithm>
using Clock = std::chrono::high_resolution_clock;

double mediana(std::vector<double> v) {
    std::sort(v.begin(), v.end());
    return v[v.size() / 2];
}

double ms_desde(Clock::time_point t0) {
    return std::chrono::duration<double, std::milli>(Clock::now() - t0).count();
}

int main() {
    const int R = 9; // repeticiones -> mediana
    std::cout << "N, filas(ms), columnas(ms), ratio, plano_filas(ms), plano_cols(ms),
ratio_plano\n";

    for (int N : {100, 500, 1000, 2000, 4000}) {
        // matriz de la guia: vector de vectores (cada fila es contigua)
        std::vector<std::vector<long long>> M(N, std::vector<long long>(N, 1LL));
        // variante adicional: un unico bloque de N*N elementos (todo contiguo)
        std::vector<long long> F((size_t)N * N, 1LL);

        std::vector<double> filas, cols, p_filas, p_cols;
        long long suma = 0, control = 0; // 'control' evita que -O2 elimine los
bucles

        for (int r = 0; r < R; r++) {
            // --- POR FILAS: j recorre memoria contigua (cache-friendly) ---
            suma = 0;
            auto t0 = Clock::now();
            for (int i = 0; i < N; i++)
                for (int j = 0; j < N; j++)
                    suma += M[i][j];
            filas.push_back(ms_desde(t0)); control += suma;

            // --- POR COLUMNAS: salto de N*8 bytes en cada acceso ---
            suma = 0;
            t0 = Clock::now();
            for (int j = 0; j < N; j++)
                for (int i = 0; i < N; i++)
                    suma += M[i][j];
            cols.push_back(ms_desde(t0)); control += suma;

            // --- Mismas pruebas sobre el vector plano ---
            suma = 0;
            t0 = Clock::now();
```

```

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            suma += F[(size_t)i * N + j];
    p_filas.push_back(ms_desde(t0)); control += suma;

    suma = 0;
    t0 = Clock::now();
    for (int j = 0; j < N; j++)
        for (int i = 0; i < N; i++)
            suma += F[(size_t)i * N + j];
    p_cols.push_back(ms_desde(t0)); control += suma;
}

double f = mediana(filas), c = mediana(cols);
double pf = mediana(p_filas), pc = mediana(p_cols);
std::cout << N << ", " << f << ", " << c << ", " << c / f
    << ", " << pf << ", " << pc << ", " << pc / pf
    // usar 'control' evita que -O2 elimine los bucles medidos:
    << " (control = " << control << ", esperado = "
    << 4LL * R * N * N << ")\n";
}
return 0;
}

```

Detalle importante sobre la variable `suma`. En el código de la guía, `suma` se reinicia antes del segundo bucle y solo se imprime al final. Al probarlo, GCC con `-O2` eliminó todos los bucles cuyo resultado se sobrescribía después: los tiempos del recorrido por filas caían a 0.000026 ms, es decir, el bucle no se ejecutaba. Por eso en nuestra versión cada suma se acumula en la variable `control`, que sí se imprime al final y cuyo valor esperado ($4 \cdot R \cdot N^2$) se verifica. Este punto se retoma en la pregunta 8.2.

6.2 Resultados

Tabla 7

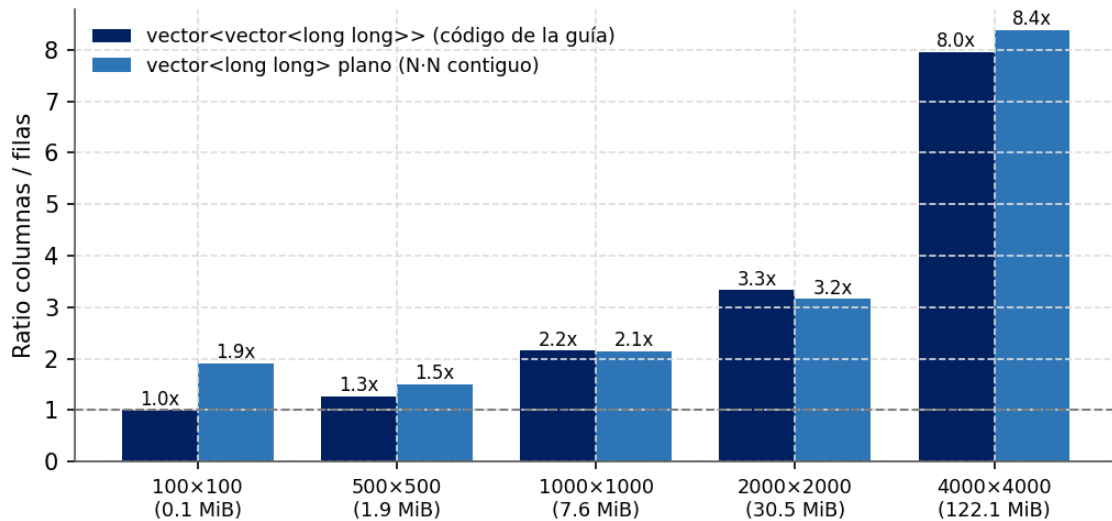
Recorrido por filas vs. por columnas de una matriz $N \times N$ de long long

Tamaño $N \times N$	Memoria	Por filas (ms)	Por columnas (ms)	Ratio (col/filas)	Ratio vector plano
100×100	78 KiB	0.004	0.004	1.00x	1.91x
500×500	1.9 MiB	0.097	0.123	1.27x	1.50x
1000×1000	7.6 MiB	0.409	0.881	2.15x	2.14x
2000×2000	30.5 MiB	4.404	14.651	3.33x	3.16x
4000×4000 *	122.1 MiB	19.347	154.031	7.96x	8.38x

Nota. Mediana de 9 repeticiones con `-O2`. (*) Tamaño adicional, no solicitado en la guía. "Ratio vector plano" corresponde a la variante con un solo `vector<long long>` de $N \cdot N$ elementos.

Figura 3

Ratio columnas/filas según el tamaño de la matriz



6.3 Análisis: cómo crece el impacto

Ambos recorridos realizan exactamente N^2 sumas, así que los dos son $\Theta(N^2)$. Sin embargo, la penalización del recorrido por columnas **crece con el tamaño**: no existe en 100×100 (1.00x), es moderada en 500×500 (1.27x), llega a 2.15x en 1000×1000, 3.33x en 2000×2000 y casi 8x en 4000×4000. La explicación está en la jerarquía de memoria:

- **100×100 (78 KiB).** Toda la matriz cabe en la caché L2 (2 MiB). Después de la primera pasada, cualquier orden de recorrido encuentra los datos en caché, por eso el ratio es 1.0.
- **500×500 (1.9 MiB).** La matriz ocupa prácticamente toda la L2. El recorrido por columnas empieza a expulsar líneas antes de reutilizarlas, lo que genera una penalización pequeña (1.27x).
- **1000×1000 y 2000×2000 (7.6 y 31 MiB).** La matriz ya no cabe en L2. Por filas, cada línea de 64 bytes aporta 8 elementos consecutivos y el prefetcher carga la siguiente antes de necesitarla. Por columnas, cada acceso salta 8 000 o 16 000 bytes: toca una línea distinta y, además, una página de memoria distinta, lo que provoca fallos de TLB (la caché de traducción de direcciones).
- **4000×4000 (122 MiB).** Cada fila mide 32 KB, de modo que el recorrido por columnas visita 4 000 páginas diferentes por columna, muchas más de las que la TLB puede recordar. El ratio sube a casi 8x.

La variante con vector plano da ratios muy parecidos a partir de 1000×1000. Por tanto, la penalización no se debe a que el `vector<vector>` de la guía guarde cada fila en un bloque separado, sino al patrón de acceso en sí. En 100×100 el vector plano muestra 1.9x, probablemente porque con `-O2` el compilador vectoriza (instrucciones SIMD) el bucle por filas sobre memoria totalmente contigua, algo que no puede hacer en el recorrido por columnas.

Conclusión de la actividad: dos algoritmos con la misma complejidad pueden diferir hasta 8 veces en tiempo real solo por el orden en que acceden a la memoria. En C/C++ el bucle interno siempre debe recorrer el último índice (`j` en `M[i][j]`).

7. Actividad 4: Python — NumPy y operaciones matriciales

7.1 Código utilizado

Se ejecutó el código de la guía con dos cambios: la multiplicación en Python puro también se midió para $n = 200$ (tarda menos de medio segundo en este equipo) y se verificó con `np.allclose` que ambos métodos producen el mismo resultado. El programa completo es el siguiente (`actividad4.py`).

```
# =====
# Practica N.o 04 - ACTIVIDAD 4 (Python)
# NumPy vs Python puro en multiplicacion de matrices
# =====

import time
import statistics
import numpy as np

def multiplicar_matrices_puro(A, B):
    """Algoritmo clasico  $O(n^3)$  con listas de listas."""
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def mediana_ms(funcion, repeticiones):
    tiempos = []
    for _ in range(repeticiones):
        t0 = time.perf_counter()
        funcion()
        tiempos.append((time.perf_counter() - t0) * 1000)
    return statistics.median(tiempos)

np.random.seed(42)
print(f"{'n':>5} {'numpy (ms)':>12} {'puro (ms)':>12} {'ratio':>10} "
      f"{'GFLOP/s np':>12} {'GFLOP/s puro':>13}")

for n in [50, 100, 200]:
    A_np = np.random.rand(n, n)
    B_np = np.random.rand(n, n)
    A_py = A_np.tolist()
    B_py = B_np.tolist()
    # lista de listas Python pura
```

```
ms_np = mediana_ms(lambda: A_np @ B_np, 50)
ms_py = mediana_ms(lambda: multiplicar_matrices_puro(A_py, B_py), 3 if n < 200
else 1)

# verificar que ambos metodos dan el mismo resultado
C_py = np.array(multiplicar_matrices_puro(A_py, B_py))
assert np.allclose(C_py, A_np @ B_np), "los resultados no coinciden"

gflops_np = 2 * n ** 3 / (ms_np / 1000) / 1e9      # 2n^3 operaciones de punto
flotante
gflops_py = 2 * n ** 3 / (ms_py / 1000) / 1e9
print(f"{n:>5} {ms_np:>12.3f} {ms_py:>12.2f} {ms_py/ms_np:>9.0f}x "
      f"{gflops_np:>12.1f} {gflops_py:>13.3f}")
```

7.2 Resultados

Tabla 8

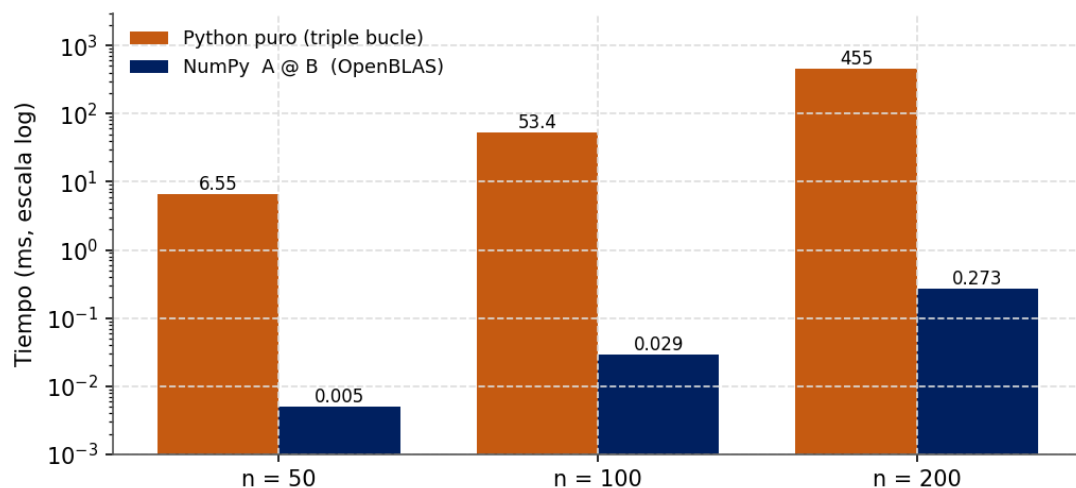
Multiplicación de matrices $n \times n$: NumPy frente a Python puro

n	NumPy (ms)	Python puro (ms)	Ratio puro/NumPy	GFLOP/s NumPy	GFLOP/s puro
50	0.005	6.55	1,308x	50.0	0.038
100	0.029	53.35	1,841x	69.0	0.037
200	0.273	455.41	1,670x	58.7	0.035

Nota. GFLOP/s = $2n^3$ / tiempo, que corresponde a las operaciones de punto flotante (multiplicaciones y sumas) del algoritmo clásico. En los tres casos `np.allclose` devolvió True.

Figura 4

Tiempo de multiplicación de matrices (escala logarítmica)



7.3 Análisis de la razón de velocidad

Ambas implementaciones ejecutan el mismo algoritmo $\Theta(n^3)$. Así lo confirma Python puro: al duplicar n , el tiempo se multiplica por 8.1 (de 50 a 100) y por 8.5 (de 100 a 200), muy cerca del

factor $2^3 = 8$ esperado. Aun así, NumPy es entre **1 300 y 1 840 veces más rápido**. La diferencia no está en la complejidad, sino en la constante, y se explica por cuatro factores:

- **Intérprete.** En Python puro, cada `C[i][j] += A[i][k] * B[k][j]` implica varias búsquedas en listas, la creación de objetos `float` y el despacho dinámico de operadores: cientos de instrucciones de máquina por cada multiplicación real.
- **Memoria contigua.** Una lista de listas guarda punteros a objetos dispersos en memoria. Un `ndarray` guarda los `double` contiguos, como el vector plano de la Actividad 3.
- **BLAS optimizado.** `A @ B` llama a OpenBLAS, que divide la matriz en bloques del tamaño de la caché (*blocking*) y usa instrucciones SIMD AVX2/FMA para operar varios números por instrucción.
- **Rendimiento medido.** NumPy alcanza entre 50 y 69 GFLOP/s en un solo núcleo, frente a 0.037 GFLOP/s de Python puro.

El ratio no crece indefinidamente con n porque ambos son $\Theta(n^3)$: la ventaja de NumPy es un **factor constante** de alrededor de 1 500x. En $n = 50$ es algo menor porque el costo fijo de llamar a BLAS (unos microsegundos) pesa más cuando el cálculo es tan pequeño. La lección práctica es que en Python los bucles numéricos anidados deben reemplazarse por operaciones vectorizadas.

8. Preguntas de análisis

8.1 ¿Cuál es el ratio exacto de crecimiento de la capacidad? ¿Es siempre 2x?

En nuestro compilador (**GCC 13.3 con libstdc++**) el ratio es **exactamente 2.0** en los 20 redimensionamientos de la Tabla 5. Sin embargo, no es siempre 2x: el estándar de C++ solo exige que `push_back` tenga costo amortizado constante, lo que se cumple con cualquier factor de crecimiento mayor que 1, y deja el valor exacto a cada implementación (cppreference.com, 2024). El factor depende de la **biblioteca estándar**, no del compilador en sí: Clang en Linux suele usar libstdc++ y en macOS usa libc++.

Tabla 9

Factor de crecimiento de `std::vector` según la biblioteca estándar

Biblioteca (compilador típico)	Factor	Redimensionamientos para 10^6	Copias totales
libstdc++ (GCC; Clang en Linux)	2.0	21 (medido)	1,048,575 (medido)
libc++ (Clang en macOS)	2.0	≈ 21	$\approx 1.05 \times 10^6$
MSVC STL (Visual Studio)	1.5	35 (simulado)	2,099,753 (simulado)

Nota. Los valores de MSVC se obtuvieron simulando su regla `nueva = anterior + anterior / 2`, que genera la secuencia 1, 2, 3, 4, 6, 9, 13, 19, 28, ... El factor 1.5 de MSVC está documentado por su propio equipo de mantenimiento (Lavavej, 2016).

Con factor 1.5 se realizan unos 14 redimensionamientos más y el doble de copias ($\approx 2n$ en lugar de $\approx n$), pero se desperdicia menos memoria y el asignador puede reutilizar bloques liberados anteriormente, algo que el factor 2 nunca permite. En ambos casos el costo amortizado sigue siendo $O(1)$.

8.2 ¿Por qué el Experimento 3 necesita la variable suma? ¿Cómo se comprueba que el benchmark es real?

Con -O2, GCC aplica *eliminación de código muerto*: si el resultado de un cálculo nunca se usa y no tiene efectos visibles, el compilador puede eliminar el bucle completo. Imprimir `suma` al final convierte el cálculo en un efecto observable y obliga a ejecutarlo. Lo comprobamos compilando el recorrido de 2000×2000 con y sin la línea que imprime `suma`:

Tabla 10

Efecto de usar o no el resultado del bucle (matriz 2000×2000, -O2)

Variante	Tiempo medido	Ancho de banda implícito	¿Es real?
Sin imprimir suma	0.000114 ms	≈ 280 000 GB/s	No: el bucle fue eliminado
Imprimiendo suma (= 4 000 000)	1.9 – 2.6 ms	≈ 12 – 17 GB/s	Sí: coherente con la RAM

Sin `suma`, 4 millones de lecturas "tardan" 0.1 microsegundos, lo cual es físicamente imposible. Para comprobar que un benchmark es real se pueden aplicar varias verificaciones, todas usadas en esta práctica:

1. **Usar el resultado:** imprimirlo o compararlo con el valor esperado (aquí `suma` debe ser N^2). También se puede usar `volatile` o `benchmark::DoNotOptimize` de Google Benchmark.
2. **Comprobar que el valor tiene sentido físico:** 32 MB leídos en unos 2 ms equivalen a unos 16 GB/s, un valor razonable para la memoria de un servidor; 280 000 GB/s no lo es.
3. **Verificar el escalado:** el tiempo debe crecer como N^2 . En la Tabla 7, pasar de 1000 a 2000 multiplica el tiempo del recorrido por filas por 10.8 (lo esperado es al menos 4x); si el bucle se hubiese eliminado, el tiempo sería el mismo para cualquier N .

En esta práctica el problema apareció de forma real: en el código original de la Actividad 3, el recorrido por filas era eliminado porque su `suma` se sobrescribía antes de imprimirse (sección 6.1). No basta, entonces, con usar el resultado en algún punto del programa: hay que usar **el resultado de cada bucle medido**.

4. **Revisar el ensamblador** con `g++ -O2 -S` o Compiler Explorer: en la versión sin `suma` no aparece el bucle de sumas (el archivo `.s` tiene 33 líneas menos).

8.3 ¿Cómo logra `std::deque` $O(1)$ en ambos extremos sin ser contiguo? ¿Cuál es su desventaja?

`std::deque` no guarda un único bloque, sino **varios bloques de tamaño fijo** (en `libstdc++`, de 512 bytes, es decir, 128 `int`) y un **mapa**: un arreglo pequeño de punteros a esos bloques que ocupa las posiciones centrales. Así funciona cada operación:

- `push_front` y `push_back`: si hay espacio en el bloque del extremo, se escribe directamente. Si no, se asigna un bloque nuevo y se añade su puntero al mapa. **Los elementos existentes nunca se copian**; solo el mapa se reasigna, de vez en cuando, cuando se queda sin posiciones libres en un extremo. Por eso el costo es $O(1)$ amortizado.
- Acceso `d[i]`: se calcula `bloque = (inicio + i) / 128` y `desplazamiento = (inicio + i) % 128`, y después se accede a `mapa[bloque][desplazamiento]`. Sigue siendo $O(1)$, pero requiere **dos direcciones** en lugar de una suma.

Desventaja frente a `std::vector` en acceso aleatorio. En nuestra medición, 1 000 000 de accesos a posiciones aleatorias tardaron **2.95 ms en deque frente a 1.62 ms en vector** (1.82x más lento). A esto se suman otras limitaciones: no ofrece un puntero contiguo `data()` para pasar a funciones de C o BLAS, el compilador no puede vectorizar los recorridos que cruzan bloques y los iteradores son más complejos. A cambio, `push_front` de un millón de elementos tardó 1.04 ms sin copiar nada, mientras que en un `vector` la inserción al inicio sería $O(n)$ por operación.

8.4 ¿Cuándo se justifica `collections.deque` en lugar de `list`? Caso de uso del curso

`list` es un arreglo dinámico como `std::vector`: `insert(0, x)` y `pop(0)` desplazan todos los elementos, con costo $O(n)$. `collections.deque` está implementada como una lista doblemente enlazada de bloques, así que `appendleft`, `popleft`, `append` y `pop` son $O(1)$. En nuestros datos (Tabla 3), 100 000 inserciones al inicio tardaron **1 050 ms con list y 5.5 ms con deque (192x)**. La contrapartida es que el acceso por índice en el centro de una deque es $O(n)$ (Python Software Foundation, 2026).

Criterio: se justifica `deque` cuando las operaciones dominantes ocurren en **los extremos** (sobre todo al inicio) y no se necesita acceso aleatorio. Se prefiere `list` cuando se accede por índice, se ordena, se hacen cortes (*slicing*) o se pasa a NumPy.

Caso concreto del curso: la implementación del TDA Cola (**FIFO**) y del recorrido en anchura (**BFS**) sobre grafos, que se estudiarán en las próximas semanas. Con `list`, `cola.pop(0)` cuesta $O(n)$ y convierte un BFS de $O(V + E)$ en uno de $O(V^2 + E)$. Con `deque.popleft()` se conserva $O(V + E)$. Otro caso ya visto en el curso es la ventana deslizante de las últimas k mediciones de un benchmark: `deque(maxlen=k)` descarta automáticamente la más antigua en $O(1)$.

9. Trabajo de investigación: algoritmo de Strassen

9.1 Idea del algoritmo

El algoritmo clásico multiplica dos matrices $n \times n$ con n^3 multiplicaciones escalares. Si cada matriz se divide en cuatro bloques de tamaño $n/2$, el método por bloques necesita 8 productos de submatrices. Strassen (1969) demostró que bastan **7 productos** si se combinan antes sumas y restas de bloques:

$M1 = (A11 + A22)(B11 + B22)$	$M5 = (A11 + A12) B22$
$M2 = (A21 + A22) B11$	$M6 = (A21 - A11)(B11 + B12)$
$M3 = A11 (B12 - B22)$	$M7 = (A12 - A22)(B21 + B22)$
$M4 = A22 (B21 - B11)$	
$C11 = M1 + M4 - M5 + M7$	$C12 = M3 + M5$
$C21 = M2 + M4$	$C22 = M1 - M2 + M3 + M6$

Se ahorra una multiplicación de bloques a cambio de 18 sumas o restas de matrices de tamaño $n/2$. Como sumar matrices cuesta $\Theta(n^2)$ y multiplicarlas $\Theta(n^3)$, el intercambio es favorable cuando n es grande.

9.2 Aplicación del Teorema Maestro

Aplicando el algoritmo de forma recursiva se obtiene la recurrencia:

$$T(n) = 7 \cdot T(n/2) + \Theta(n^2)$$

El Teorema Maestro resuelve recurrencias de la forma $T(n) = a \cdot T(n/b) + f(n)$ comparando $f(n)$ con $n^{\log_b a}$ (Cormen et al., 2022). Aquí:

- $a = 7$ subproblemas, $b = 2$ (cada uno de tamaño $n/2$) y $f(n) = \Theta(n^2)$, que corresponde a las 18 sumas de bloques: $18 \cdot (n/2)^2 = 4.5 \cdot n^2$.
- Exponente crítico: $\log_2 7 \approx 2.807$, por lo que $n^{\log_b a} = n^{2.807}$.
- Como $f(n) = \Theta(n^2) = O(n^{(2.807 - \epsilon)})$ con $\epsilon \approx 0.8 > 0$, se cumple el **Caso 1**: el costo está dominado por las hojas de la recursión.

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

Para comparar, el método por bloques con 8 productos da $T(n) = 8T(n/2) + \Theta(n^2)$ y, por el mismo Caso 1, $\Theta(n^{\log_2 8}) = \Theta(n^3)$. **Toda la ganancia asintótica proviene de reducir a de 8 a 7.** En cada nivel de recursión el trabajo de multiplicación se reduce en un factor $7/8$: con k niveles se realiza $(7/8)^k$ del trabajo original.

9.3 ¿A partir de qué tamaño supera al algoritmo $O(n^3)$?

Análisis teórico por conteo de operaciones. Un nivel de Strassen seguido del algoritmo clásico en los bloques requiere $7 \cdot (2(n/2)^3 - (n/2)^2) + 18 \cdot (n/2)^2 = 1.75n^3 + 2.75n^2$ operaciones, frente a $2n^3 - n^2$ del clásico. Strassen gana cuando $0.25n^3 > 3.75n^2$, es decir, para $n > 15$. Sin embargo, este conteo ignora el costo de la memoria, de las matrices temporales y de la recursión, por lo que el punto de inflexión real es mucho mayor.

Evidencia de la literatura. Higham (1990) señaló que Strassen resulta útil en la práctica una vez que las dimensiones superan aproximadamente 100, y analizó su estabilidad numérica dentro de BLAS nivel 3. Huss-Lederman et al. (1996) concluyeron que el umbral de corte óptimo depende de la arquitectura y que conviene detener la recursión en bloques medianos. Huang et al. (2016) mostraron que, integrando las sumas de Strassen en el empaquetado de datos de BLIS, se obtienen aceleraciones incluso en matrices relativamente pequeñas y rectangulares, y Huang et al. (2020) trasladaron este enfoque a GPU.

Experimento propio. Se implementó Strassen en C++17 (-O2) con un **umbral de corte** n_0 : cuando el bloque tiene tamaño n_0 o menos, se usa el algoritmo clásico con orden de bucles i-k-j, que es caché-amigable según lo visto en la Actividad 3. Se comparó con el mismo algoritmo clásico aplicado a la matriz completa. El código completo se encuentra en `strassen.cpp`; se transcriben aquí las tres funciones centrales.

```
// Multiplicacion clasica O(n^3) con orden de bucles i-k-j (cache-friendly).
// la, lb, lc son los "anchos de fila" de cada bloque dentro de su matriz madre.
void clasico(const double* A, const double* B, double* C, int n, int la, int lb, int lc) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) C[i * lc + j] = 0;
        for (int k = 0; k < n; k++) {
            double a = A[i * la + k];
```

```
        for (int j = 0; j < n; j++) C[i * lc + j] += a * B[k * lb + j];
    }
}

// C = A + s*B (s = 1 suma, s = -1 resta) sobre bloques de tamaño n x n.
void combinar(const double* A, int la, const double* B, int lb, double* C, int n, double s) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i * n + j] = A[i * la + j] + s * B[i * lb + j];
}

// Algoritmo de Strassen con umbral de corte: si n <= corte se usa el clasico.
void strassen(const double* A, const double* B, double* C,
              int n, int la, int lb, int lc, int corte) {
    if (n <= corte) { clasico(A, B, C, n, la, lb, lc); return; }

    int h = n / 2; // tamaño de cada bloque
    size_t s = (size_t)h * h;
    const double *A11 = A, *A12 = A + h,
                  *A21 = A + h * la, *A22 = A + h * la + h;
    const double *B11 = B, *B12 = B + h,
                  *B21 = B + h * lb, *B22 = B + h * lb + h;

    std::vector<double> T1(s), T2(s), M[7];
    for (auto& m : M) m.assign(s, 0.0);

    // Los 7 productos de Strassen (en lugar de los 8 del metodo por bloques)
    combinar(A11, la, A22, lb, T1.data(), h, 1); // M1 = (A11+A22)(B11+B22)
    combinar(B11, lb, B22, lb, T2.data(), h, 1);
    strassen(T1.data(), T2.data(), M[0].data(), h, h, h, h, corte);

    combinar(A21, la, A22, lb, T1.data(), h, 1); // M2 = (A21+A22) B11
    strassen(T1.data(), B11, M[1].data(), h, h, lb, h, corte);

    combinar(B12, lb, B22, lb, T2.data(), h, -1); // M3 = A11 (B12-B22)
    strassen(A11, T2.data(), M[2].data(), h, la, h, h, corte);

    combinar(B21, lb, B11, lb, T2.data(), h, -1); // M4 = A22 (B21-B11)
    strassen(A22, T2.data(), M[3].data(), h, la, h, h, corte);

    combinar(A11, la, A12, lb, T1.data(), h, 1); // M5 = (A11+A12) B22
    strassen(T1.data(), B22, M[4].data(), h, h, lb, h, corte);

    combinar(A21, la, A11, lb, T1.data(), h, -1); // M6 = (A21-A11)(B11+B12)
    combinar(B11, lb, B12, lb, T2.data(), h, 1);
    strassen(T1.data(), T2.data(), M[5].data(), h, h, h, h, corte);
}
```

```
combinar(A12, 1a, A22, 1a, T1.data(), h, -1); // M7 = (A12-A22)(B21+B22)
combinar(B21, 1b, B22, 1b, T2.data(), h, 1);
strassen(T1.data(), T2.data(), M[6].data(), h, h, h, h, corte);

// Reconstruccion de los cuatro bloques del resultado
for (int i = 0; i < h; i++)
    for (int j = 0; j < h; j++) {
        size_t k = (size_t)i * h + j;
        C[i * 1c + j] = M[0][k] + M[3][k] - M[4][k] + M[6][k]; // C11
        C[i * 1c + j + h] = M[2][k] + M[4][k]; // C12
        C[(i + h) * 1c + j] = M[1][k] + M[3][k]; // C21
        C[(i + h) * 1c + j + h] = M[0][k] - M[1][k] + M[2][k] + M[5][k]; // C22
    }
}
```

Tabla 11

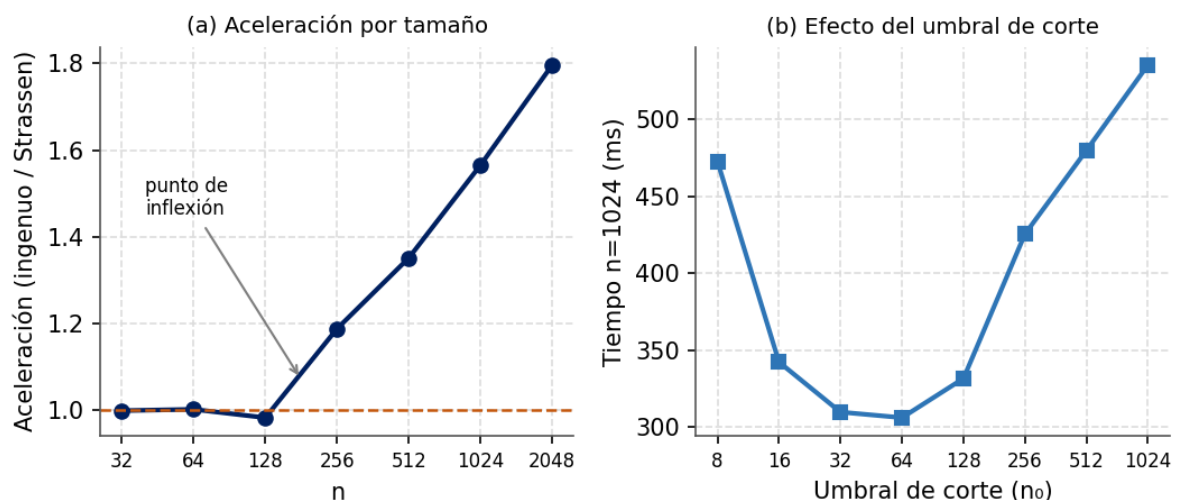
Algoritmo clásico $O(n^3)$ vs. Strassen con umbral de corte $n_0 = 64$ (C++17, -O2)

n	Clásico (ms)	Strassen (ms)	Aceleración	Error máx. $ C_1 - C_2 $
32	0.013	0.013	1.00x	0
64	0.096	0.096	1.00x	0
128	0.824	0.839	0.98x	7.8e-14
256	7.7	6.5	1.19x	3.1e-13
512	63.0	46.6	1.35x	1.4e-12
1024	512.3	327.3	1.57x	6.3e-12
2048	4098.9	2281.6	1.80x	2.8e-11

Nota. Para $n \leq 64$ ambos ejecutan el mismo código (el caso base), por eso la aceleración es 1.00. El error máximo, del orden de 10^{-11} para valores de magnitud ~ 500 , es propio del redondeo de punto flotante.

Figura 5

Aceleración de Strassen por tamaño (a) y efecto del umbral de corte con $n = 1024$ (b)



Resultado: en este equipo el punto de inflexión empírico está entre $n = 128$ y $n = 256$. Con $n = 128$, Strassen todavía es ligeramente más lento (0.98x); con $n = 256$ ya es 1.19x más rápido, y la

ventaja crece con n : 1.35x con 512, 1.57x con 1024 y 1.80x con 2048. Este crecimiento coincide con el conteo de multiplicaciones: con $n_0 = 64$ y $n = 2048$ se realizan $7^5 \cdot 64^3 \approx 4.41 \times 10^9$ multiplicaciones frente a $2048^3 \approx 8.59 \times 10^9$, una reducción teórica de 1.95x. La aceleración medida (1.80x) es algo menor porque las sumas de bloques y las matrices temporales tienen un costo real.

La Figura 5(b) muestra por qué no conviene aplicar la recursión hasta bloques de 1×1 : con $n_0 = 8$, el costo de las llamadas y de las sumas supera al ahorro (472 ms); el óptimo está en $n_0 \approx 32-64$ (306–310 ms), y con $n_0 = 1024$ se vuelve al algoritmo clásico (535 ms).

9.4 Limitaciones prácticas

Nuestro Strassen es 1.8x más rápido que el algoritmo clásico propio, pero NumPy/OpenBLAS multiplica una matriz de 2048×2048 en unos 243 ms, casi 10 veces menos que nuestro Strassen (2 282 ms), usando el algoritmo $O(n^3)$. Esto confirma la idea central de la práctica: **una constante pequeña, lograda con caché, SIMD y memoria contigua, puede pesar más que un exponente menor** en los tamaños habituales. Por eso las bibliotecas BLAS comerciales usan casi siempre el algoritmo clásico optimizado. Además, Strassen requiere memoria adicional para las matrices temporales, funciona de forma natural solo con n potencia de 2 (en otro caso hay que rellenar con ceros) y tiene cotas de error normwise algo más débiles que el algoritmo clásico, aunque sigue siendo numéricamente estable (Demmel et al., 2007; Higham, 1990). La investigación continúa: con aprendizaje por refuerzo, AlphaTensor descubrió algoritmos que multiplican matrices de 4×4 con 47 productos en aritmética módulo 2, mejorando los 49 que se obtienen aplicando Strassen dos veces (Fawzi et al., 2022).

10. Discusión de limitaciones

Los resultados se obtuvieron en una máquina virtual con un solo núcleo. Los valores absolutos cambiarán en otros equipos, aunque las tendencias (lineal frente a cuadrático, duplicación de la capacidad, crecimiento del ratio de caché) deberían mantenerse. Los tiempos inferiores a 0.01 ms, como el recorrido de 100×100 o NumPy con $n = 50$, están cerca de la resolución del reloj y deben interpretarse con cautela; por eso se reporta la mediana de varias repeticiones. La caché L3 de 260 MiB que informa la VM es compartida con otros procesos del servidor físico, lo que puede explicar parte de la variabilidad observada entre corridas (en los tamaños grandes puede superar el 30 % entre corridas). Por último, el análisis de caché y TLB de la sección 6.3 se infiere de los tiempos medidos; confirmarlo directamente requeriría contadores de hardware como `perf stat`.

11. Conclusiones

1. **El acceso por índice es $O(1)$ y la inserción al inicio es $O(n)$.** El acceso `lst[i]` tardó ~ 55 ns para cualquier n y posición. En cambio, `insert(0)` creció de forma cuadrática en el total, hasta ser 231 veces más lento que `append` con $n = 100\,000$. La diferencia supera un orden de magnitud desde $n \approx 5\,000$.
2. **La duplicación hace que `push_back` sea $O(1)$ amortizado.** GCC duplicó la capacidad en los 21 redimensionamientos (ratio exacto 2.0) y realizó 1 048 575 copias para un millón de

inserciones (1.05 por elemento, por debajo de la cota $2n$). `reserve()` eliminó esas copias y aceleró el proceso 1.25x. MSVC usa factor 1.5, que implica el doble de copias pero menos memoria desperdiciada.

3. **La localidad de caché cambia la constante, no la complejidad.** Con el mismo $\Theta(N^2)$, el recorrido por columnas pasó de no tener penalización (100×100) a ser 3.3x más lento en 2000×2000 y casi 8x en 4000×4000 , a medida que la matriz dejó de caber en la caché.
4. **El código vectorizado domina en Python.** NumPy fue entre 1 300 y 1 840 veces más rápido que el triple bucle en Python puro, con la misma complejidad $\Theta(n^3)$, gracias a la memoria contigua, BLAS y SIMD.
5. **Strassen es $\Theta(n^{2.807})$ por el Caso 1 del Teorema Maestro.** En la práctica superó al algoritmo clásico propio a partir de $n \approx 256$ (1.8x con $n = 2048$), con un umbral de corte óptimo de 32–64. Aun así, un BLAS $O(n^3)$ optimizado fue unas 10 veces más rápido, lo que confirma que los detalles de implementación pueden pesar más que el exponente asintótico.

12. Declaración de uso de inteligencia artificial

En cumplimiento de las normas de integridad académica del curso, se declara el uso de herramientas de inteligencia artificial generativa en la elaboración de este trabajo:

- **Herramienta:** Claude (Anthropic).
- **Uso:** apoyo en la adaptación del código de la guía (repeticiones, contador de copias, bucle de tamaños), en la ejecución de los benchmarks, en la generación de las figuras y en la redacción preliminar del análisis y de la investigación sobre Strassen.
- **Responsabilidad:** los integrantes del equipo revisaron el código, verificaron los resultados, las citas y las referencias, y asumen la responsabilidad total del contenido presentado.

13. Referencias

- Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/>
- cppreference.com. (2024). *std::vector — C++ reference*. <https://en.cppreference.com/w/cpp/container/vector>
- Demmel, J., Dumitriu, I., Holtz, O., & Kleinberg, R. (2007). Fast matrix multiplication is stable. *Numerische Mathematik*, 106(2), 199–224. <https://doi.org/10.1007/s00211-007-0061-6>
- Fawzi, A., Balog, M., Huang, A., Hubert, T., Romera-Paredes, B., Barekatin, M., Novikov, A., Ruiz, F. J. R., Schrittwieser, J., Swirszcz, G., Silver, D., Hassabis, D., & Kohli, P. (2022). Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930), 47–53. <https://doi.org/10.1038/s41586-022-05172-4>
- Goodrich, M. T., Tamassia, R., & Mount, D. M. (2011). *Data structures and algorithms in C++* (2nd ed.). Wiley.

- Higham, N. J. (1990). Exploiting fast matrix multiplication within the level 3 BLAS. *ACM Transactions on Mathematical Software*, 16(4), 352–368. <https://doi.org/10.1145/98267.98290>
- Huang, J., Smith, T. M., Henry, G. M., & van de Geijn, R. A. (2016). Strassen's algorithm reloaded. En *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC16)* (pp. 690–701). IEEE. <https://doi.org/10.1109/SC.2016.58>
- Huang, J., Yu, C. D., & van de Geijn, R. A. (2020). Strassen's algorithm reloaded on GPUs. *ACM Transactions on Mathematical Software*, 46(1), Artículo 1. <https://doi.org/10.1145/3372419>
- Huss-Lederman, S., Jacobson, E. M., Johnson, J. R., Tsao, A., & Turnbull, T. (1996). Implementation of Strassen's algorithm for matrix multiplication. En *Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*. IEEE. <https://doi.org/10.1145/369028.369096>
- Lavavej, S. T. (2016, 11 de octubre). *[libcxx] [test] Fix non-Standard assumptions about how many elements are allocated* (D25483) [Revisión de código]. LLVM Phabricator. <https://reviews.llvm.org/D25483>
- NumPy Developers. (2024). *NumPy documentation*. <https://numpy.org/doc/stable/>
- Python Software Foundation. (2026). *collections — Container datatypes*. Python 3 documentation. <https://docs.python.org/3/library/collections.html>
- Strassen, V. (1969). Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4), 354–356. <https://doi.org/10.1007/BF02165411>